

3) An image shows aliasing artifacts. Identify the cause using sampling and quantization concepts and propose a corrective approach.

SOLUTION:

Answer:

Aliasing Artifacts: Cause and Corrective Approach

Cause (Using Sampling and Quantization Concepts):

Aliasing occurs when an image is **sampled at a rate lower than the required sampling frequency (Nyquist rate)**. If the image contains high-frequency details (such as thin lines, edges, or repetitive patterns), these details cannot be represented correctly with too few pixels.

As a result:

Fine patterns appear as **jagged edges (staircase effect)**.

Repeating patterns may create **false patterns (moiré effect)**.

Image details become distorted or inaccurate.

Sampling Concept:

The image is converted from a continuous signal into discrete pixels.

If the **sampling frequency is too low**, high-frequency information is lost, causing aliasing.

Quantization Concept:

Quantization converts continuous intensity values into a finite number of gray levels.

Although coarse quantization mainly causes **banding or false contours**, it can make aliasing artifacts appear more noticeable by reducing intensity precision.

Corrective Approach

Increase the Sampling Rate

Capture or scan the image at a higher resolution.

Ensure the sampling frequency is at least twice the highest frequency present in the image (Nyquist criterion).

Apply an Anti-Aliasing (Low-Pass) Filter

Blur or smooth the image slightly before sampling. This removes high-frequency components that cause aliasing.

Use Higher Image Resolution

More pixels preserve fine details and reduce jagged edges.

Use Better Interpolation During Resizing

Apply **bilinear**, **bicubic**, or **Lanczos interpolation** instead of nearest-neighbor interpolation to reduce aliasing during scaling.

Conclusion

Aliasing is primarily caused by **insufficient sampling**, where the sampling rate is too low to capture high-frequency image details. Quantization can further reduce image quality by limiting intensity levels. The best solution is to **increase the sampling rate, apply anti-aliasing filters before sampling, and use higher-resolution imaging with appropriate interpolation methods.**

CODE:

```
import cv2

import matplotlib.pyplot as plt

# Load image

img = cv2.imread('image.jpg')

# Convert BGR to RGB

img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

# -----

# Create aliasing by shrinking and enlarging

# -----

small = cv2.resize(img, (100, 100), interpolation=cv2.INTER_NEAREST)

aliased = cv2.resize(small, (img.shape[1], img.shape[0]),
```

```

        interpolation=cv2.INTER_NEAREST)

# -----

# Corrective approach:

# Apply Gaussian Blur (Anti-Aliasing)

# -----

blurred = cv2.GaussianBlur(img, (5, 5), 0)

# Resize using better interpolation

corrected = cv2.resize(blurred, (img.shape[1], img.shape[0]),

        interpolation=cv2.INTER_CUBIC)

# -----

# Display Results

# -----

plt.figure(figsize=(15,5))

plt.subplot(1,3,1)

plt.imshow(img)

plt.title("Original Image")

plt.axis("off")

plt.subplot(1,3,2)

plt.imshow(alias)

plt.title("Aliased Image")

plt.axis("off")

plt.subplot(1,3,3)

plt.imshow(corrected)

plt.title("Corrected (Anti-Aliased)")

plt.axis("off")

```

```
plt.tight_layout()
```

```
plt.show()
```